

LocoBot Fleet — How the Bots Decide

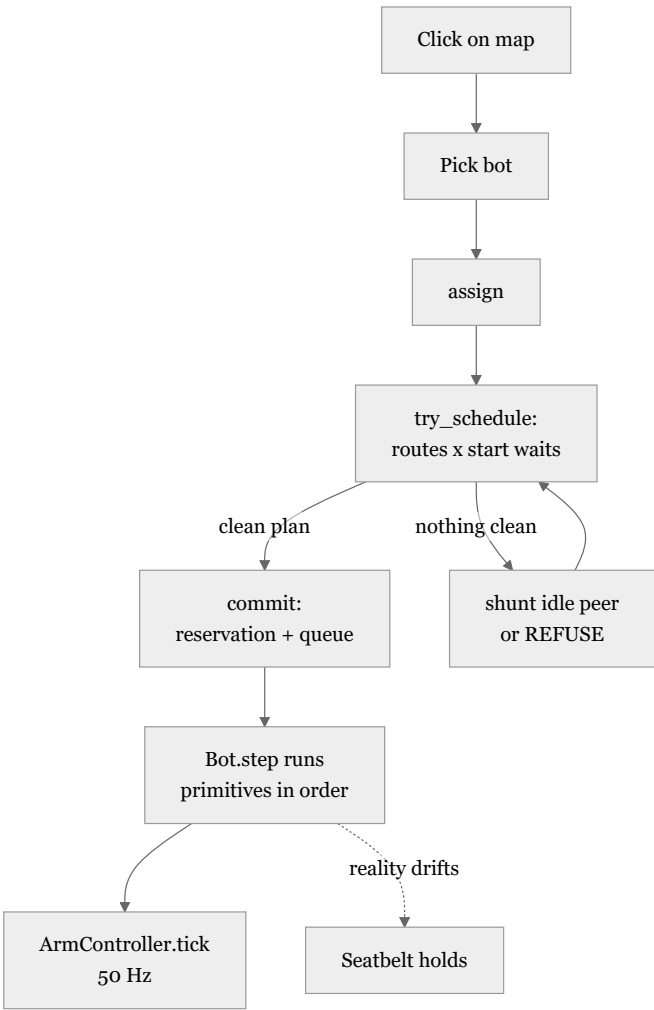
Sep 24, 2026 · @Sai Teja

1. The big picture

A bot never reacts its way to a target. It plans the whole trip in advance, checks that plan against the other bot's committed plan in space **and** time, and only moves once the plan is conflict-free. Everything lives in `run_fleet2.py`, with the arm stack in `arm_control.py`, `keepout.py` and `arm_log.py`.

There are four layers, and each has one job:

Layer	Code	When it runs	Job
Planner	<code>assign()</code> , <code>try_schedule()</code> , <code>simulate_plan()</code> , <code>conflict()</code>	Once, when you click	Pick a route and a start delay that never overlap the other bot
Executor	<code>Bot.step()</code>	Every physics step (2 ms)	Run the plan's steps (primitives) in order
Seatbelt	checks inside <code>Bot.step()</code>	Every physics step	Stop the bot if reality drifts from the plan; counts <code>reflex_holds</code>
Arm controller	<code>ArmController.tick()</code>	50 Hz	Move the arm along smooth trajectories, supervise it, enforce keep-out zones



The planner decides; the executor obeys; the seatbelt is insurance. A working schedule keeps the seatbelt count at **zero**, so any non-zero `seatbelt=` in the map title means the plan and reality disagreed.

2. Glossary

These are the words the code, comments and on-screen messages use. The code name is what to search for.

The world

Term	Code name	Meaning
Lane A / Lane B	LANE_A, LANE_B	The two rail lines the bots drive along, at $y = 0.231$ m and 0.551 m
Station / turntable	ST_LEFT, ST_RIGHT	The only places a bot can change lanes, at $x = 0.211$ m and 1.251 m. A rail segment rotates 90° with the bot on it
Station zone	STATION_ZONE	± 0.27 m around a station. While a turntable turns, no other bot may be inside it
Charging bay	green area, $x > 1.42$ m	Parking area at the right end of both lanes

Plans

Term	Code name	Meaning
Primitive	prim	One step of a plan: DRIVE_X, DRIVE_Y, TURN, WAIT or ARM
Plan / route	plan, queue	An ordered list of primitives, e.g. drive to a station, turn, cross, turn back, drive on
Candidate route	cands	One of the 2–3 routes the planner considers for a trip
Direct route	plan_route()	The shortest way: straight along the lane, or via the nearest-sensible station if lanes differ
Detour route	detour_route()	A same-lane trip that goes round via the <i>other</i> lane, using both stations
Start wait	("WAIT", w) at the front	Seconds the bot waits before starting, tried from 0 to 24 s in 1 s steps
Samples	samples	The plan turned into a timeline of (t, x, y) points every 0.25 s
Reservation	plan_samples, plan_locks, plan_discs	A committed plan: where the bot will be, which stations it will lock, where its arm will be out, and when
Station lock	locks	(station, t_start, t_end): the window a bot's turn occupies a turntable, padded by T_BUF
Work disc / arm capsule	discs	A 3D segment + thickness + time window: the space the extended arm occupies
Conflict	conflict()	The first place-and-time where two reservations overlap, or none
Shunt	inside assign()	Moving an idle bot out of the way to make room, planned with the same checks
Refused	"NO CLEAN SCHEDULE"	No route, wait or shunt produced a conflict-free plan
String diagram	bottom panel	x position vs time for each bot's plan; overlapping lines mean a clash

Execution and safety

Term	Code name	Meaning
Executor	Bot.step()	Runs the current primitive; start_next() moves to the next

Term	Code name	Meaning
Seatbelt / reflex hold	<code>reflex_holds</code>	A live safety stop (arm out while driving, peer in the way, turntable busy). Should stay 0
Stowed (certified)	<code>stowed()</code>	Arm reach under 0.13 m; stays certified until reach exceeds 0.19 m

Arm

Term	Code name	Meaning
Pose	<code>POSES["STOW"]</code> , <code>POSES["WORK"]</code>	Named sets of 5 joint angles
Move / trajectory	<code>Move</code>	Smooth cubic path from one pose to another; all joints share one duration <code>T</code>
Tick	<code>tick()</code>	One 20 ms controller cycle
Wanted / sent	<code>wanted</code> , <code>sent</code>	Angle the plan wants vs angle actually written (wanted + sag, clamped)
Sag	<code>sag</code>	The droop gravity causes; corrected by adding it in advance (feed-forward)
Status	<code>status</code>	<code>ok</code> , <code>lag</code> (trailing), <code>blocked</code> (stuck, trajectory paused), <code>keepout</code> (would enter a zone)
Keep-out zone	<code>keepout.ZONES</code>	A box in world or bot coordinates the arm may never enter
Refused move	<code>refused</code>	A move rejected because its path enters a keep-out zone

3. How the planner decides, step by step

The planner tries every candidate route with every start wait from 0 to 24 s, throws out any combination that clashes with the other bot, and keeps the one that **finishes earliest**. It runs once per click, in `assign()` and `try_schedule()`.

Step 0: before planning (`on_press`)

- **Which bot?** If you selected one with 1 / 2, that one. In AUTO (0), the bot with the smallest $|x - \text{target } x|$, plus a 0.5 m penalty if it is busy.
- **Target refused outright** if it is within 0.28 m of a station (the turntable's swept area) or within 0.26 m of the other bot's destination on the same lane.

Step 1: build candidate routes (`try_schedule`)

Situation	Candidates tried
Already on the target lane	1. Direct: drive straight along the lane 2. Detour left-first: cross at the left station, drive on the other lane, cross back at the right station 3. Detour right-first: the mirror image
On the other lane	1. Via the "natural" station: the one minimising `

If the arm is out, a ("ARM", "STOW") goes first. A work task (right-click) adds ARM WORK → WAIT 8 s → ARM STOW at the end.

Step 2: turn each plan into a timeline (`simulate_plan`)

Each primitive gets a duration, and the bot's position is sampled every 0.25 s (DT):

Primitive	Duration	Also records
DRIVE_X / DRIVE_Y	distance ÷ 0.13 m/s + 0.6 s settle	position along the way
TURN	3.0 s + 0.5 s	a station lock from 1.5 s before to 1.5 s after
ARM	computed move time T + 0.8 s	a work disc from the extend to the stow, ±1.5 s
WAIT	as given	position held

Step 3: check against the other bot (`conflict`)

The other bot's committed reservation is the obstacle. If it is idle, that is just its parked spot (plus its arm if the arm is out). The check walks time from now to 8 s past the later plan's end, and at each 0.25 s step asks:

Conflict code	Meaning	Rule
body	The bots would get too close	Both `
lockA / lockB	A bot is near a turntable the other is turning	Within 0.27 m of the station during its lock window
armA / armB	An extended arm would hit the other bot's chassis or its hanging stowed arm	3D distance between arm segment and body
locklock	Both plans turn the same station at overlapping times	Time windows overlap
armarm	Both arms out and touching	3D segment distance under the two thicknesses

Conflict code	Meaning	Rule
armlock	An extended arm sits inside a turntable's swing while it turns	x-extent overlap with the lock window

The ± 1.5 s check on `body` means a bot keeps a time gap as well as a distance gap.

Step 4: search waits, keep the earliest finish

```

for each candidate route:
  for w in 0, 1, 2 ... 24 s:
    plan = [stow if needed] + [WAIT w] + route + [work steps]
    if no conflict:
      if this plan ends earlier than the best so far: keep it
      stop trying longer waits for this route

```

The winner is the plan with the **earliest end time** (`tend`). Distance is not scored anywhere. A longer route wins whenever it finishes sooner, usually because the short route would have to wait.

Step 5: if nothing is clean, try a shunt (`assign`)

This only happens when the other bot is **idle**. The planner looks for a parking spot to move it to:

- Spots at $x = 0.60, 0.90, 1.60, 0.35, 1.05$ m, on the other bot's other lane first, then its own lane.
- A spot is skipped if it is within 0.30 m of a station, 0.35 m of your target, or 0.35 m of you.
- For each spot: plan the shunt, commit it, then re-plan your trip. If your trip still fails, undo the shunt and try the next spot.

Step 6: commit or refuse

A clean plan is committed: its timeline, locks and discs become this bot's reservation, and its primitives become the queue. If nothing works you see **NO CLEAN SCHEDULE**, and the bot does not move.

4. Why the bot didn't take the shortest path

The bot does not look for the *shortest* path. It looks for the **earliest-finishing plan that never clashes** with the other bot. The short route loses whenever it would clash at every start wait from 0 to 24 s, or when waiting for it would finish later than going round.

$$\text{chosen} = \arg \min_{\text{route}, w \in \{0, 1, \dots, 24\}} t_{\text{end}}(\text{route}, w) \quad \text{subject to no conflict}$$

So there are only three reasons a bot avoids the shortest route:

1. **The short route is never clean.** Something permanent sits in its way, usually the other bot parked, or finishing its plan parked, on the path.
2. **The short route needs a station that is locked.** The other bot is turning there, and its plan ends somewhere that keeps blocking.
3. **Waiting would take longer than going round.** Possible in principle, but rare: a detour costs about 35–45 s, and a wait is capped at 24 s.

Worked examples (real planner output)

These were produced by running the actual `try_schedule()` on set-up positions. The times are seconds from the click.

Example A: going round a parked bot. bot1 at $x = 0.60$ on lane A, target $x = 1.60$ on lane A. bot2 is parked, idle, at $x = 0.95$ on lane A, right in the way.

Route	Distance	Time with no wait	First clash at wait 0	Result
Direct	1.00 m	8.3 s	body at 0 s	Never clean
Detour, left station first	2.42 m	35.6 s	none	Chosen , ends at 35.6 s
Detour, right station first	3.72 m	45.6 s	body at 0 s	Never clean (it must drive through bot2 to reach the right station)

The shortest route is $2.4\times$ shorter but can never pass bot2, which is parked forever. The planner didn't shunt bot2, because shunting only happens when *no* route works.

Example B: using the far station. bot1 at $x = 0.50$ on lane B wants $x = 0.60$ on lane A. bot2 has just started crossing at the left station, from lane A to lane B, and will park at $x = 0.30$ on lane B.

Route	Distance	Time with no wait	First clash at wait 0	Result
Via left station (natural)	1.00 m	16.5 s	lockB at 0.2 s	Never clean
Via right station	1.72 m	22.0 s	none	Chosen , ends at 22.0 s

bot1 is already inside the left station's zone while bot2 turns it. Waiting doesn't help either: bot2 ends parked at $x = 0.30$ on lane B, between bot1 and the left station.

Example C: a near-tie decided by a lock. bot1 at $x = 0.90$ on lane A wants $x = 0.50$ on lane B. bot2 is crossing at the left station and will park at $x = 0.45$ on lane A.

Route	Distance	Time with no wait	First clash at wait 0	Result
Via left station (natural)	1.30 m	18.8 s	lockB at 3.2 s	Never clean

Route	Distance	Time with no wait	First clash at wait o	Result
Via right station	1.42 m	19.7 s	none	Chosen , ends at 19.7 s

Only 0.12 m and 0.9 s separate the two routes. On the screen this looks like the bot "went the wrong way". In fact bot2 will park at $x = 0.45$ on lane A, which is inside the left station's 0.27 m zone and in bot1's path to it.

Example D: waiting beats going round. bot1 at $x = 0.60$ on lane A wants $x = 1.00$ on lane A. bot2 is crossing at the right station from lane B to lane A. It holds that station from 1.8 s to 14.8 s, then drives into the bay.

Route	Distance	Time with no wait	First clash at wait o	Result
Direct	0.40 m	3.7 s	lockB at 3.0 s	Clean with a 13 s wait, chosen , ends at 16.7 s
Detour, left station first	2.32 m	34.8 s	none	Clean at once, but ends at 34.8 s
Detour, right station first	3.12 m	41.0 s	lockB at 3.0 s	Clean with a 13 s wait, ends at 54.0 s

The target $x = 1.00$ is 0.25 m from the right station, inside its 0.27 m zone, so bot1 can't arrive while bot2 is turning. Here a clean detour exists, but the short route plus a 13 s wait finishes 18 s earlier. This is why a bot sometimes just sits there before moving.

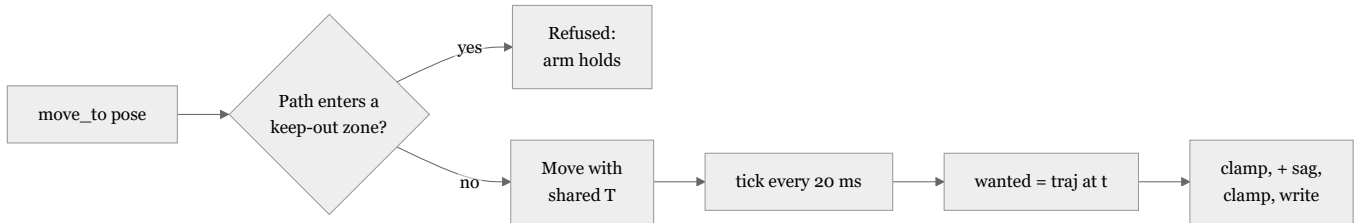
How to tell which reason applies

- **Look at the string diagram** (bottom panel). Amber bands are station locks and purple bands are arm discs. If the other bot's line ends flat across your bot's straight path, it's reason 1.
- **A flat start on your bot's line** is a start wait (Example D).
- **An amber band at the natural station** during your bot's trip is reason 2 (Examples B and C).
- **Check the other bot's final parking spot.** Most "never clean" results come from where the other bot *ends*, not where it is now. The planner treats the other bot's last position as held forever (`pos_at` holds endpoints).

5. How the arm moves

The arm never jumps to a pose. Every move is a smooth cubic path that all five joints finish together, sent to the servos 50 times a second, watched by a supervisor, and checked against keep-out zones before and during the move.

One move, from command to motion



1. **Plan the move** (`move_to`). Take the last commanded pose, the target pose and the limits, and compute one duration T for all joints. The slowest joint sets T , using the peak speed and acceleration of the cubic curve, not its average.
2. **Check the path**. Sample the path about every 0.02 rad of joint motion. If any sample puts an arm link inside a keep-out zone, refuse. The arm keeps holding, and `refused` names the zones.
3. **Run it** (`tick`, every 20 ms). Work out where the path says the arm should be *now*, measured by the clock rather than by counting ticks, so a late tick catches up by itself. Then clamp to soft limits, add the sag correction, clamp to the servo range, and write all five joints at once.
4. **Supervise**. Compare each joint's measured angle with the wanted angle:
 - `lag`: more than 0.20 rad behind for 0.3 s while still moving. This is reported only.
 - `blocked`: more than 0.15 rad off *and* not moving for 0.3 s. The path's clock pauses, and it resumes when the joint frees up.
 - `keepout`: the next wanted pose would enter a zone. The arm holds and the clock pauses.
5. **Finish**. `done()` is true once the path's clock passes T and nothing is blocking it.

Formulas

The cubic path, where $s = t / T$ runs from 0 to 1:

$$\theta(s) = A + (B - A)(3s^2 - 2s^3)$$

The shared duration, taken over all joints j :

$$T = \max_j \left(\frac{1.5|\Delta\theta_j|}{v_{max,j}}, \sqrt{\frac{6|\Delta\theta_j|}{a_{max,j}}} \right)$$

For STOW to WORK, the shoulder moves 1.2 rad, which gives $T = 1.34$ s. The peak speed seen in the sim was 1.39 rad/s, against a 1.5 rad/s limit.

How the arm fits into fleet plans

- An ("ARM", `pose`) step calls `move_to` when it starts. The planner budgets the same computed T plus 0.8 s (`ARM_PAD`), so plan and executor agree.
- **WORK** counts as finished when the path is done and joints 2–4 are within 0.12 rad of the pose. If it's still not finished after $T + 3$ s, it gives up and the plan moves on.

- **STOW** counts as finished only when the path is done *and* reach stays under 0.13 m for 0.6 s. It never times out: a jammed stow holds the bot in place rather than letting it drive with the arm out.
- A refused WORK is skipped, so the arm stays stowed. A refused STOW is held as a fault.

The joint log

Press 1 in the window, or start with `--log`, to write `logs/arm_joints_<date>_<time>.csv`. You get one row per bot, 50 times a second, with:

- the time, current step and supervisor status
- the world x/y/z of every joint and of the gripper tip
- each joint's measured angle (`_q`) and wanted angle (`_qw`)
- the keep-out zones the arm is actually inside, if any

6. Variables reference

Every tunable number, what it does, and which decision it affects. Distances are in metres, times in seconds and angles in radians unless marked otherwise. Two different constants are both called `V_MAX`: the bot's drive speed in `run_fleet2.py`, and the arm's joint speeds in `arm_control.py`.

World and planning (`run_fleet2.py`)

Name	Value	Meaning	Affects
<code>LANE_A</code> , <code>LANE_B</code>	0.231, 0.551	Lane y positions	Routes
<code>ST_LEFT</code> , <code>ST_RIGHT</code>	0.211, 1.251	Station x positions	Routes, locks
<code>X_MIN</code> , <code>X_MAX</code>	0.09, 1.70	Clickable target range	Target clamp
<code>V_MAX</code>	0.13 m/s	Planned drive speed	Every drive time
<code>SETTLE</code>	0.6	Extra time after each drive	Plan length
<code>TURN_TIME</code>	3.0	Turntable 90° turn (planned as 3.5 with margin)	Lock length
<code>DEG90</code>	1.5708	Turntable crossing angle	Turns
<code>STATION_ZONE</code>	0.27	Keep-clear radius around a turning station	<code>lockA</code> / <code>lockB</code>
<code>BODY_CLEAR</code>	0.29	Minimum bot–bot gap in both x and y	<code>body</code>
<code>T_BUF</code>	1.5	Time padding on bodies, locks and discs	Every conflict
<code>DT</code>	0.25	Timeline sampling step	Plan and check resolution
<code>HORIZON_PAD</code>	8.0	How far past the end conflicts are checked	Late clashes
<code>WORK_DWELL</code>	8.0	Time at the target in a work task	Work plan length
<code>ARM_PAD</code>	0.8	Added to each arm move's computed <code>T</code>	Arm step length
<code>POSES</code>	STOW (0, 0.3, -0.6, 0, 0), WORK (0, -0.9, 0.6, -0.2, 0)	Named arm poses (5 joints)	Arm steps, work discs
<code>L1</code> , <code>L2</code> , <code>L3</code>	0.1357, 0.1861, 0.245	Arm links for the <i>collision envelope</i> . <code>L3</code> is padded; the real wrist-to-tip distance is 0.1548	Arm discs, reach
<code>ARM_HW</code>	0.06	Half-width of the arm capsule (link + sway)	<code>arm*</code> conflicts
<code>SEG_SWEEP</code>	0.146	Turntable segment swing radius	<code>armlock</code>
<code>Z_BASE</code>	0.145	Shoulder height (world z)	Arm envelope
<code>CH_R</code> , <code>CH_Z</code>	0.17, (0.23, 0.49)	Other bot's chassis as a vertical capsule	<code>armA</code> / <code>armB</code>
<code>COL_R</code> , <code>COL_Z</code>	0.14, (-0.31, 0.23)	Other bot's hanging stowed arm as a column	<code>armA</code> / <code>armB</code>
<code>WORK_R</code> , <code>STOW_R_MAX</code>	0.30, 0.15	Defined but not used by any logic	—

Numbers written directly in the code

Where	Value	Meaning
<code>try_schedule</code>	0 to 24 s, 1 s steps	Start waits tried per route

Where	Value	Meaning
on_press	0.28	Targets this close to a station are refused
on_press	0.26	Targets this close to the other bot's destination are refused
on_press (AUTO)	+0.5	Penalty added to a busy bot when choosing who goes
assign	x = 0.60, 0.90, 1.60, 0.35, 1.05	Shunt parking spots, tried in this order
assign	0.30 / 0.35 / 0.35	Shunt spot must be this far from stations / your target / you
stowed()	0.13 / 0.19	Reach to certify stowed / to revoke it
Bot.step DRIVE	gain 3.0, arrive at 0.004	Drive speed = $3 \times \text{error}$, capped at V_MAX
Bot.step seatbelt	0.24 across, 0.32 along	Peer too close ahead: hold
Bot.step ARM	0.12; 0.6 s; T + 3 s	WORK settle tolerance; STOW calm window; WORK timeout

Arm controller (`arm_control.py`, `keepout.py`)

Name	Value	Meaning
SC15_COUNT	0.205°	One servo step ($210^\circ \div 1024$)
V_MAX (arm)	1.5, 1.5, 1.5, 2.0, 2.0 rad/s	Joint speed limits, placeholders until bench tests
A_MAX	4, 4, 4, 6, 6 rad/s ²	Joint acceleration limits, placeholders
SOFT_MARGIN	0.05	Soft limits sit this far inside the hard range
rate_hz	50	Controller tick rate
profile	cubic	Move shape (quintic available)
BLOCK_ERR, BLOCK_SPEED, BLOCK_TIME	0.15, 0.05, 0.3	When a joint counts as blocked
LAG_ERR, LAG_TIME	0.20, 0.3	When a joint counts as lagging
MARGIN (keep-out)	0.010	Clearance added to each link's radius
LINK_R	0.032, 0.028, 0.025	Upper arm, forearm, wrist + gripper radii
STEP	0.005	Sampling spacing along each link

Servo model (`model_fleet.xml`, `class` `sc15`)

Attribute	Value	Meaning
kp	20	Servo stiffness, a guess until the bench sag test
forcerange	±1.67 N·m	17 kg·cm at 8.4 V
dampratio	1	Critically damped: no ringing
armature	0.01	Gearbox-reflected rotor inertia

7. "Why did my bot do X?" checklist

Match what you saw to the most likely cause and where to look. Most surprises come from the other bot's *final* parking spot, a turntable lock, or the 24 s cap on waiting.

What you saw	Most likely cause	Where to look
Went the long way round	The short route clashes at every wait from 0 to 24 s	String diagram: the other bot's line ends across your path (section 4, Example A)
Crossed at the far station	The natural station is locked, or the other bot parks near it	Amber band at the natural station (Examples B and C)
Sat still, then went	A start wait was cheaper than any detour	Flat start on its string-diagram line (Example D)
The <i>other</i> bot moved when you clicked	Shunt: nothing worked, and the other bot was idle	<code>assign()</code> shunt spots
"NO CLEAN SCHEDULE"	No route, wait or shunt was clean; or the other bot is busy, so it can't be shunted	Try again when the other bot is idle, or pick a target further from it
"REFUSED: inside a turntable's swept zone"	Target within 0.28 m of a station	Click further from the shaded bands
"REFUSED: that spot is botN's destination"	Within 0.26 m of the other bot's target on the same lane	Pick another spot
<code>seatbelt</code> count above 0	Reality drifted from the plan (arm not stowed, peer closer than planned, turntable busy)	Which primitive was running; the arm title shows ARM OUT
Bot stuck in an ARM step	STOW can't certify: reach isn't staying under 0.13 m, or the stow was refused by a keep-out zone	Arm panel title: status and reach
Arm didn't extend on a work task	WORK refused by a keep-out zone	The message names the zone
Arm shows LAG	Trailing more than 0.20 rad while moving	Normal trailing is about 9° at full speed. Lower the arm <code>V_MAX</code> if it persists
Arm shows BLOCKED	A joint is off target and not moving; the path is paused	Remove the obstruction and it resumes
Arm shows KEEPOUT	The next step of the path would enter a zone	<code>keepout.ZONES</code> ; the log's <code>keepout</code> column
Slider move ignored	Refused by a keep-out zone (the slider keeps your value)	The message; move the slider back

A quick experiment for any route question

1. Pause with the space bar before clicking, so both bots stay where they are.
2. Note where the other bot is **and where its plan ends** (the × marker on the map).
3. Ask: does the short route pass within 0.29 m of that end point, or within 0.27 m of a station the other bot turns while you'd be near it? If yes, the short route is dead, and the planner picks the earliest-finishing alternative.